5 주차 보고서

전공: 컴퓨터공학과 학년: 4 학년 학번: 20232019 이름: Maksutova Aibike

Problem 1: Who is the Richest?

1. Idea

We can only ask "does person i have at least x won?". With this question we can do binary search on one person's money. Money is at most 10^9, and 2^30 > 10^9, so we need about 30 questions to know the exact money of one person.

The simple way is to do binary search for every person and take the biggest. But this needs 30 * N questions, which is too many. We don't really need to know the exact money of everyone. We only need the exact money of the richest person.

So my idea is to keep the current richest person (I call him the champion) and his exact money V. For every other person I ask only one question: "do you have at least V+1 won?". If the answer is No, this person is not richer, so I skip him. If the answer is Yes, he is the new champion, and I do binary search only for him to find his exact money.

2. Algorithm

1. Shuffle the people in random order.
2. Do binary search on the first person in [0, 10^9]. Now he is the champion and V = his money.
3. For each next person i, ask "at least V+1?".
   - No: skip person i.
   - Yes: binary search person i in [V+1, 10^9], then champion = i, V = his money.
4. At the end print the champion and V.

```
best = p[0]; V = binary_search(p[0], 0, 1e9)
for i in p[1..N-1]:
    if ask(i, V+1) == Yes:
        best = i; V = binary_search(i, V+1, 1e9)
print(best, V)
```

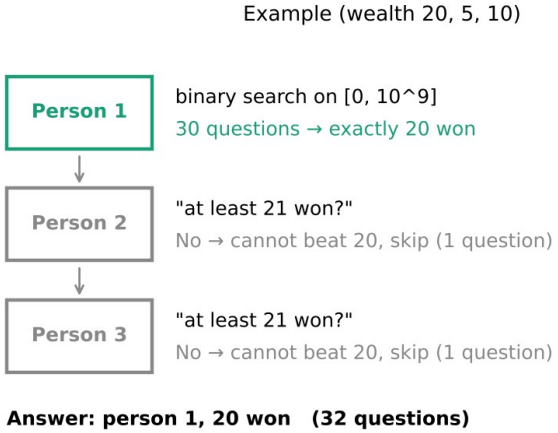


Figure 1. How the algorithm works on the example.

In the example, person 1 needs 30 questions (we get 20 won). Then for person 2 and person 3 we ask "at least 21?" and both say No. So the answer is person 1 with 20 won, and we used only 32 questions instead of 90.

This is correct because a No answer means the person has at most V won, so he can't be richer than the champion. A Yes answer means he has more than V, so he becomes the new richest. If two people have the same money, we keep the first one, and the problem says any of them is OK.

3. Why shuffle?

Most people cost only 1 question. Only when the champion changes we pay about 30 more questions. If the people are sorted by money from small to big, the champion changes every time and we again need about 30 * N questions. If we shuffle first, the k-th person is the richest so far only with probability 1/k, so the champion changes only about ln N times on average. So the total is about N + 30 * ln N questions. For example, for N = 100000 it is around 100 360 questions, compared with 3 000 000 for the simple way.

We can't do much better than this, because every person except the answer must be asked at least once (otherwise maybe he is the richest), and we also need around 30 questions to find the exact amount. So about N + 30 is the minimum.

Time complexity: O(N + 30 log N) on average. Memory: O(N).

Problem 2: Chasing Light

1. Understanding the problem

Every second the robot either moves 1 cell or stays. If it stays on a cell with light, it gets 1 energy. I understood the light interval [S, E] as E - S seconds (seconds S, S+1, ..., E-1), because then the example gives 9. To go from one light to another the robot needs |x1 - x2| + |y1 - y2| seconds (Manhattan distance).

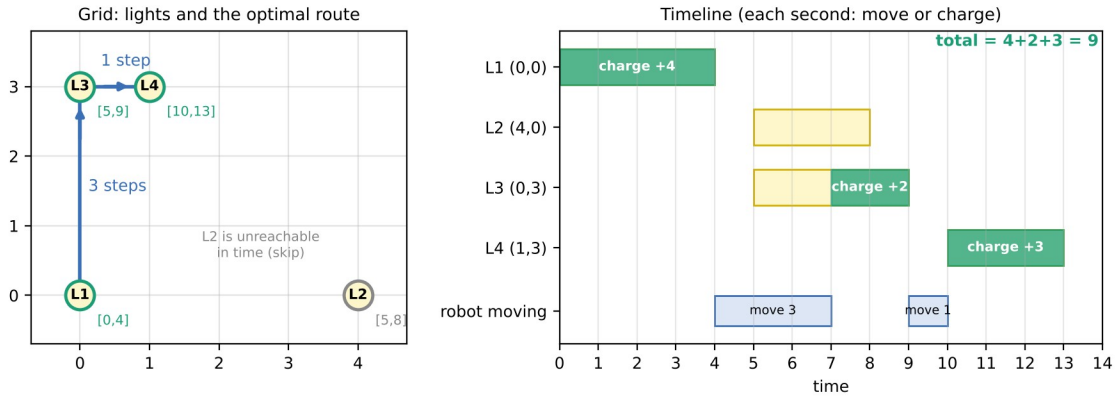


Figure 2. Best route for the example: 4 + 2 + 3 = 9.

In the example: the robot charges at L1 for 4 seconds (time 0~4), moves 3 seconds to L3 (arrives at 7), charges 2 seconds (7~9), moves 1 second to L4 (arrives at 10), charges 3 seconds (10~13). Total = 9. L2 is too far, so we skip it.

2. Main observation

If the robot is charging at some light, it can always stay there until the light ends (time E). Leaving early never helps.

Why: if the robot leaves light j δ seconds earlier, it loses δ energy at j. It arrives at the next light δ seconds earlier, so it can get at most δ more energy there (or 0 if that light has not started yet). So staying until E is never worse (Figure 3).

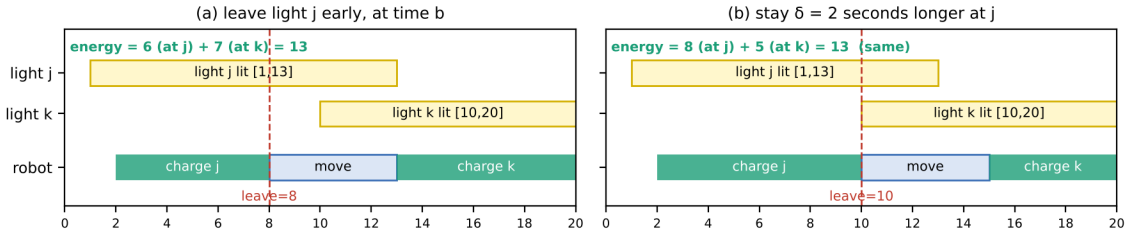


Figure 3. Leaving 2 seconds later: +2 at light j, -2 at light k, the total is the same.

Because of this, the robot's plan is just a list of lights, and at each light it stays until the end. If it arrives at light j at time a, it gets E_j - max(S_j, a) energy there (if it arrives early, it just waits).

3. DP

- Add a fake light 0 at (0, 0) with S = E = 0 (the start).
- Sort all lights by E.
- dp[j] = max energy when the robot is at light j at time E_j.
- From light i the robot arrives at j at time a = E_i + dist(i, j). If a <= E_j:

  dp[j] = max(dp[j], dp[i] + E_j - max(S_j, a))

- Answer = max of all dp[j].

We sort by E because the robot can go from i to j only if E_j >= E_i + dist.

Example: dp[L1] = 4, dp[L2] = 4 (it arrives at L2 at time 8 when the light is already off, so +0), dp[L3] = dp[L1] + 2 = 6 (arrives at 7), dp[L4] = dp[L3] + 3 = 9 (arrives at 10). Answer = 9.

4. Complexity

Sorting takes O(N log N) and the DP checks every pair (i, j), so the time is O(N^2). Memory is O(N). I also wrote a brute force (simulate every second) and compared it with the DP on many small random tests, and the answers were the same.